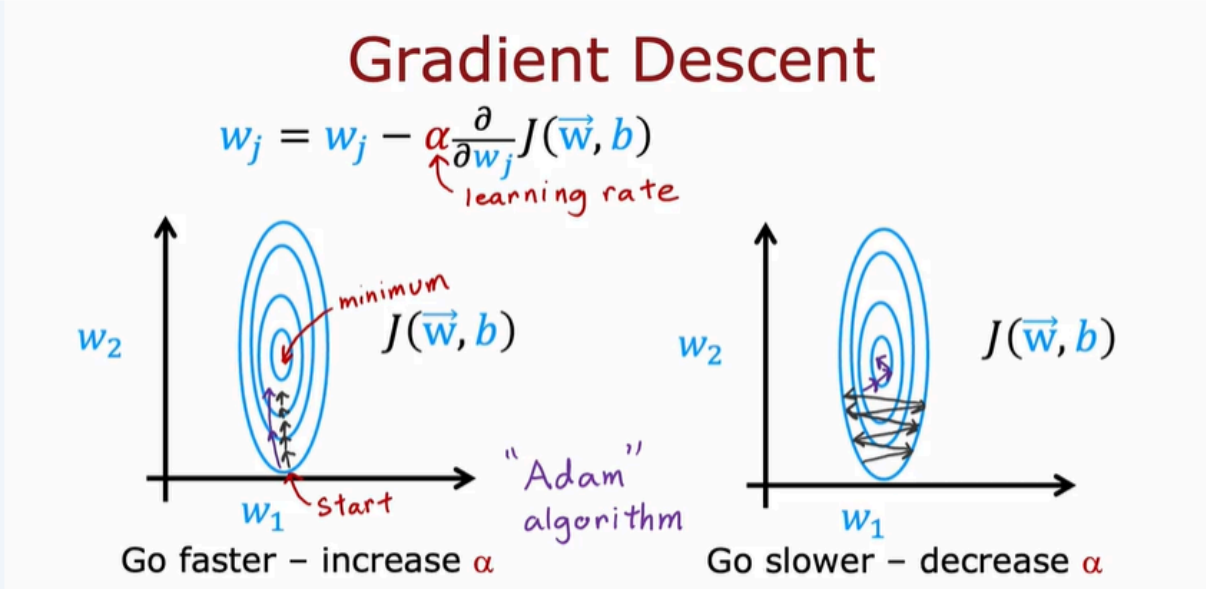
# Additional Neural Network Concepts

## Advanced Optimization



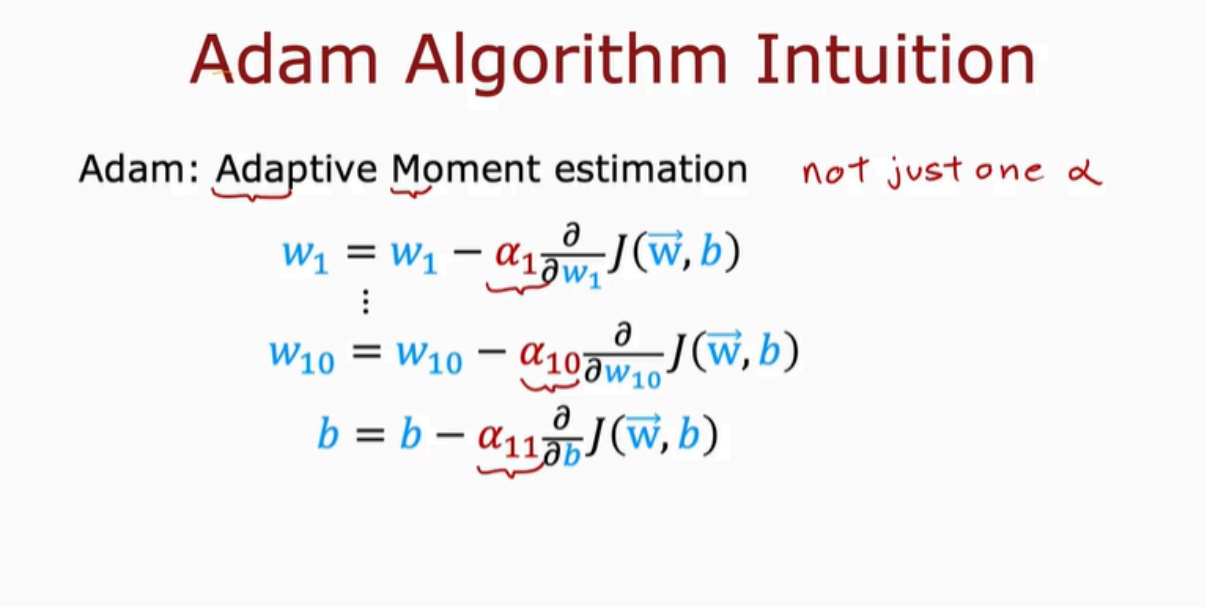

**Gradient descent** is an optimization algorithm that is widely used in machine learning, and was the foundation of many algorithms like linear regression and logistic regression and early implementations of neural networks.

- But it turns out that there are now some other optimization algorithms for minimizing the cost function, that are even better than gradient descent. In this video, we'll take a look at an algorithm that can help you train your neural network much faster than gradient descent.
- Recall that this is the expression for one step of gradient descent. A parameter w_j is updated as w_j minus the learning rate Alpha times this partial derivative term. How can we make this work even better?
- In this example, I've plotted the cost function J using a contour plot comprising these ellipsis, and the minimum of this cost function is at the center of this ellipsis down here. Now, if you were to start gradient descent down here, one step of gradient descent, if Alpha is small, may take you a little bit in that direction. Then another step, then another step, then another step, then another step, and you notice that every single step of gradient

descent is pretty much going in the same direction, and if you see this to be the case, you might wonder, well, why don't we make Alpha bigger, can we have an algorithm to automatically increase Alpha? They just make it take bigger steps and get to the minimum faster. There's an algorithm called the Adam algorithm that can do that. If it sees that the learning rate is too small, and we are just taking tiny little steps in a similar direction over and over, we should just make the learning rate Alpha bigger.

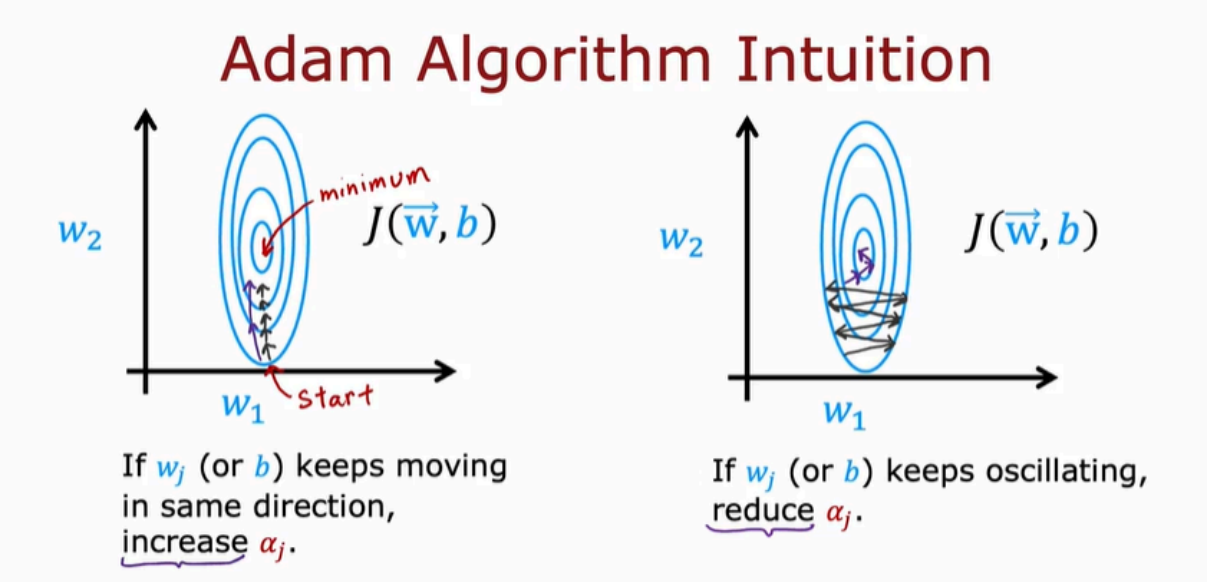
- In contrast, here again (the image in the right), is the same cost function if we were starting here and have a relatively big learning rate Alpha, then maybe one step of gradient descent takes us here, in the second step takes us here, third step, and the fourth step, and the fifth step, and the sixth step, and if you see gradient descent doing this, is oscillating back and forth. You'd be tempted to say, well, why don't we make the learning rates smaller? The Adam algorithm can also do that automatically, and with a smaller learning rate, you can then take a more smooth path toward the minimum of the cost function. Depending on how gradient descent is proceeding, sometimes you wish you had a bigger learning rate Alpha, and sometimes you wish you had a smaller learning rate Alpha.

## Adam Algorithm

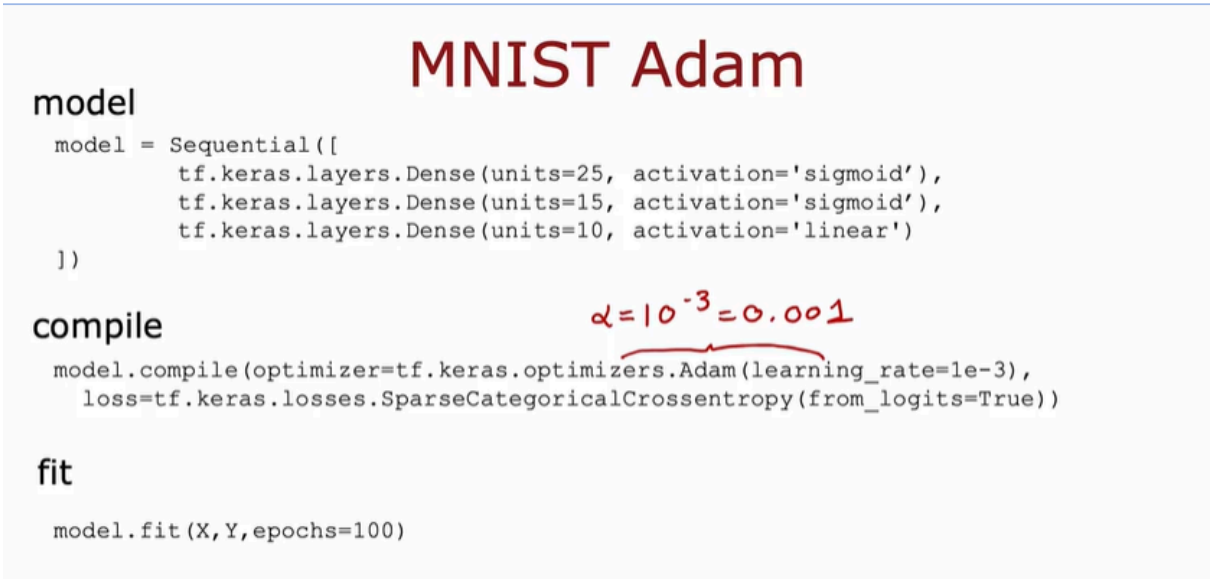



The Adam algorithm can adjust the learning rate automatically. Adam stands for Adaptive Moment Estimation, or A-D-A-M, and don't worry too much about what this name means, it's just what the authors had called this algorithm. But

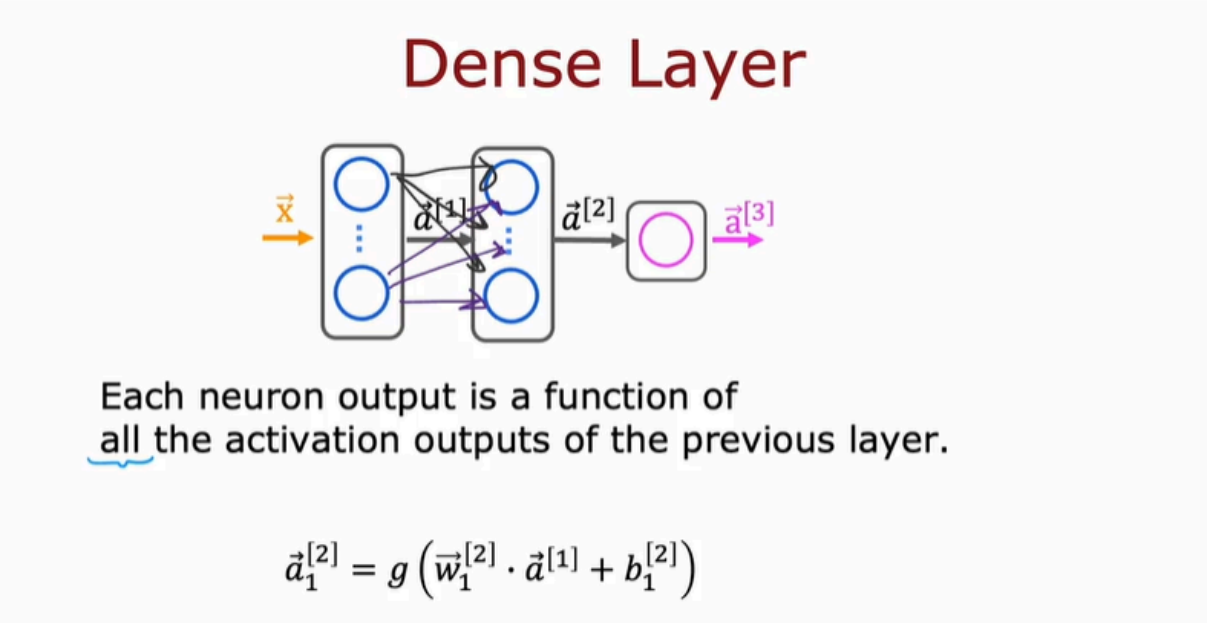
interestingly, the Adam algorithm doesn't use a single global learning rate Alpha. It uses a different learning rates for every single parameter of your model. If you have parameters $w_1$ through $w_{10}$, as was b, then it actually has 11 learning rate parameters, $\alpha_1$, $\alpha_2$, all the way through $\alpha_{10}$ for $w_1$ to $w_{10}$, as well as I'll call it $\alpha_{11}$ for the parameter b.




The implement



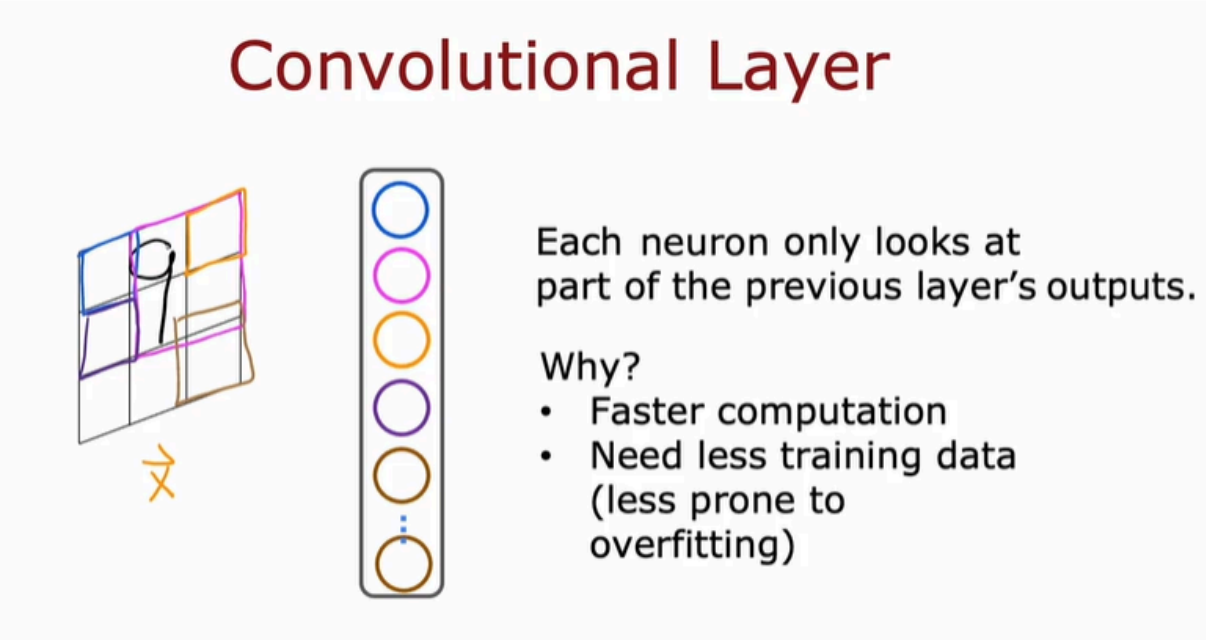

# Additional Layer Types




All the neural network layers with you so far have been the dense layer type in which every neuron in the layer gets its inputs all the activations from the previous layer. And it turns out that just using the dense layer type, you can actually build some pretty powerful learning algorithms. And to help you build further intuition about what neural networks can do.

It turns out that there's some other types of layers as well with other properties. In this video I'd like to briefly touch on this and give you an example of a different type of neural network layer. Let's take a look to recap in the dense layer that we've been using the activation of a neuron in say the second hidden layer Is a function of every single activation value from the previous layer of a one. But it turns out that for some applications, someone designing a neural network may choose to use a different type of layer.

## Convolutional Layer

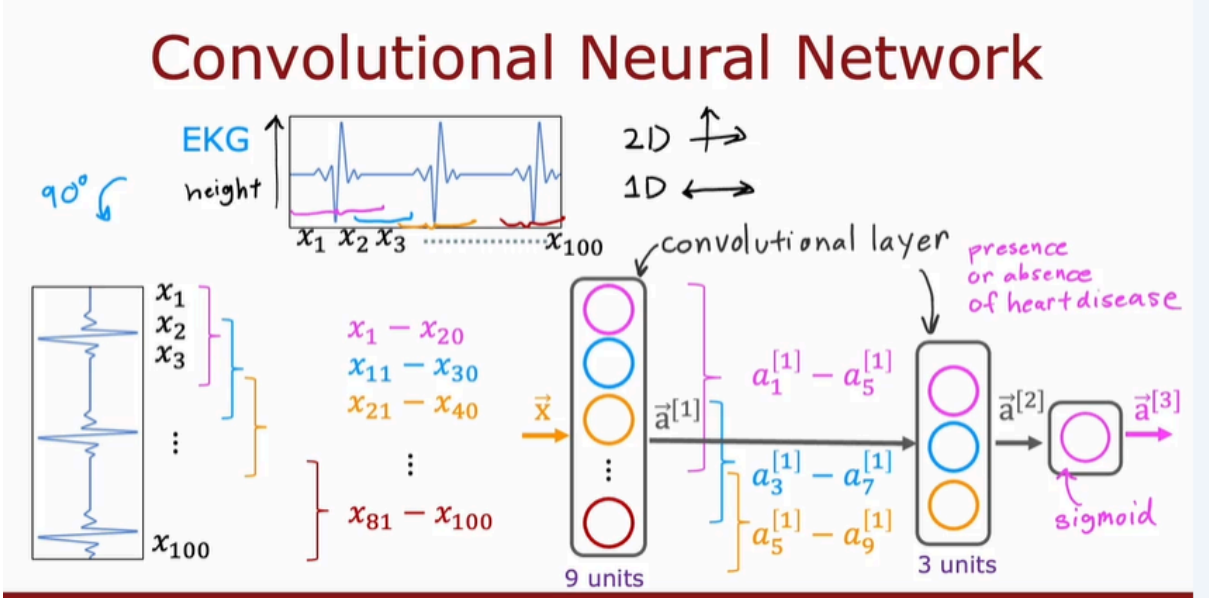



One other layer type that you may see in some work is called a convolutional layer. Let me illustrate this with an example. So what I'm showing on the left is the input X. Which is a handwritten digit nine. And what I'm going to do is construct a hidden layer which will compute different activations as functions of this input image X. But here's something I can do for the first hidden unit, which I've drawn in blue rather than saying this neuron can look at all the pixels in this image. I might say this neuron can only look at the pixels in this little rectangular region. Second neuron, which I'm going to illustrate in magenta is also not going to look at the entire input image X instead, it's only going to look at the pixels in a limited region of the image. And so on for the third neuron and the 4th neuron and so on and so forth. Down to the last neuron which may be looking only at that region of the image.

**So why might you want to do this? Why won't you let every neuron look at all the pixels but instead look at only some of the pixels?**

- Well, some of the benefits are first, it speeds up computation.
- And second advantage is that a neural network that uses this type of layer called a convolutional layer can need less training data or alternatively, it can also be less prone to overfitting.
    - You heard me talk a bit about overfitting in their previous course but this is something that will dive into greater detail on next week as well. When we talk about practical tips for using learning algorithms and this is the type of layer where each neuron only looks at a region of

the input image is called a convolutional layer. It was a researcher John Macoun who had figured out a lot of the details of how to get convolutional layers to work and popularized their use.




Let me illustrate in more detail a convolutional layer. And if you have multiple convolutional layers in a neural network sometimes that's called a convolutional neural network. To illustrate the convolutional layer of convolutional neural network on this slide I'm going to use instead of a two D image input. I'm going to use a one dimensional input and the motivating example I'm going to use is classification of E K G signals or electrocardiograms. So if you put two electrodes on your chest you will record the voltages that look like this (the image of heart signals) that correspond to your heartbeat. This is actually something that my stanford research group did research on. We were actually reading E K G signals that actually look like this to try to diagnose if patients may have a heart issue. So an E K G signal and electoral cardia graham E C G in some places E K G.

In some places there's just a list of numbers corresponding to the height of the surface at different points in time. So you may have say 100 numbers corresponding to the height of this curve at 100 different points of time. And the learning tosses given this time series, given this E K G signal to classify say whether this patient has a heart disease or some diagnosable heart conditions. Here's what the convolutional neural network might do. So I'm going to take the E K G signal and rotated 90 degrees to lay it on the side. And so we have here 100 inputs X_1, X_2 all the way through X_100. Like. So and

when I construct the first hidden layer Instead of having the first hidden unit take us input all 100 numbers. Let me have the first hidden unit. Look at only X_1 through X_20. So that corresponds to looking at just a small window of this E K G signal.

The second hidden unit shown in a different color here. Well look at X_11 through X_30, so looks at a different window in this E K G signal. And the third hidden there looks at another window X_21 through X_40 and so on. And the final hidden units in this example.

Well Look at X_81 through X_100. So it looks like a small window towards the end of this EKG time series. So this is a convolutional layer because these units in this layer looks at only a limited window of the input. Now this layer of the neural network has nine units. The next layer can also be a convolutional layer.

So in the second hidden layer let me architect my first unit not to look at all nine activations from the previous layer, but to look at say just the first 5 activations from the previous layer. And then my second unit In this second hidden there may look at just another five numbers, say A3-A7. And the third and final hidden unit in this layer will only look at A5 through A9.

And then maybe finally these activations. A2 gets inputs to a sigmoid unit that does look at all three of these values of A2 in order to make a binary classification regarding the presence or absence of heart disease. So this is the example of a neural network with the first hidden layer being a convolutional layer. The second hidden layer also being a convolutional layer and then the output layer being a sigmoid layer. And it turns out that with convolutional layers you have many architecture choices such as how big is the window of inputs that a single neuron should look at and how many neurons should layer have. And by choosing those architectural parameters effectively, you can build new versions of neural networks that can be even more effective than the dense layer for some applications.